ÉCOLE MAROCAINE DES SCIENCES DE L'INGÉNIEUR — EMSI
5IASD — Déploiement d'IA (2026-2027)

Rapport de TP04 : AutoML
Explorer les plateformes AutoML et MLOps low-code

Étudiant : Amine KHABOT

Filière : 5IASD

Dépôt GitHub : https://github.com/Amine-kh1/TP04_AutoML_LowCode

Environnement : Python 3.13.5 | FLAML 2.7.0 | MLflow 3.17.2

1. Introduction et Objectifs du TP

L'objectif de ce travail pratique est de comparer trois paradigmes d'obtention de modèles prédictifs sur une tâche de détection de fraudes bancaires (5 000 transactions, cible déséquilibrée) :

- Un modèle de référence simple et transparent : Régression Logistique scikit-learn.
- Un modèle réglé manuellement au TP2 : Forêt Aléatoire (Random Forest 500 arbres, profondeur 16).
- Une recherche AutoML locale automatisée : FLAML (Fast and Lightweight AutoML) évaluée sur 4 budgets de calcul (10s, 30s, 60s, 120s).

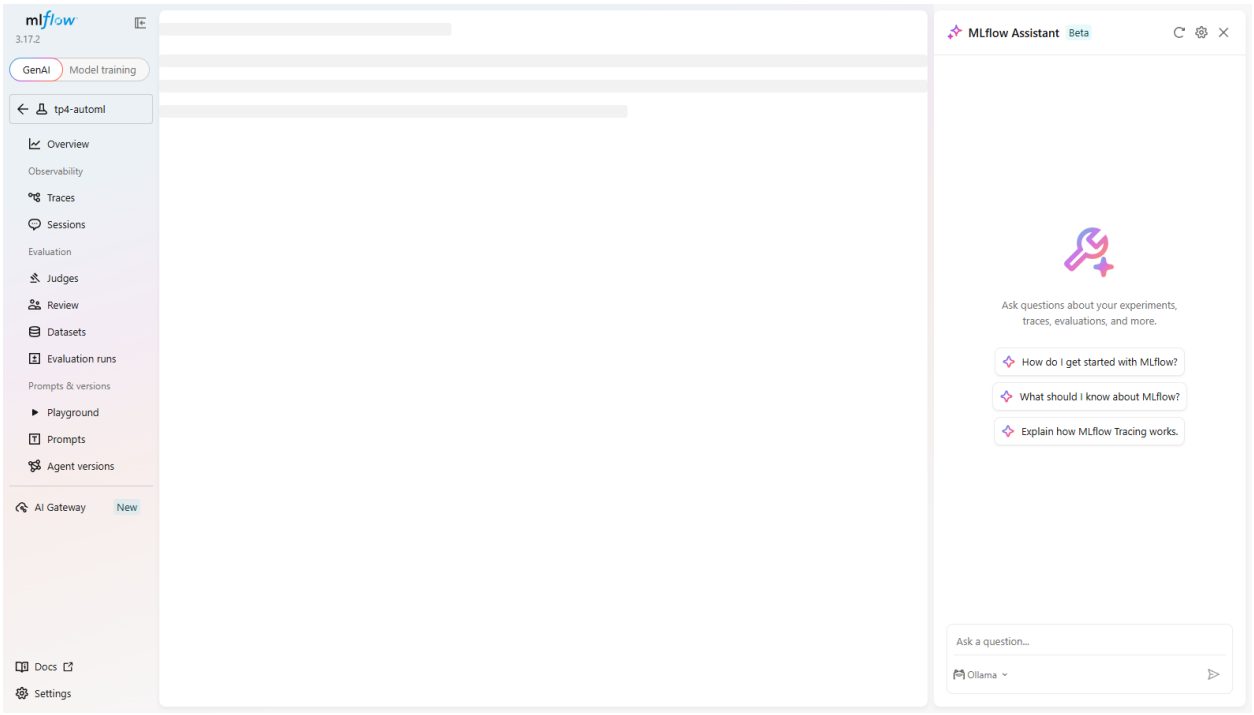
Le projet évalue également la capacité d'inspection des essais (auditabilité), la vulnérabilité de l'AutoML face au piège critique de la fuite de données (data leakage), ainsi qu'une comparaison détaillée avec les plateformes cloud managées du marché.

2. Bloc A : Établissement des Modèles de Référence

Afin de juger la performance d'un outil AutoML, deux points de repère ont été entraînés et tracés dans MLflow (expérience 'tp4-automl') avec la même graine aléatoire et le même découpage train/test (75% / 25%) :

Approche	Modèle / Hyperparamètres	Score F1 Test	Temps Entraînement
reference-logistique	LogisticRegression(max_iter=1000)	0.7407	0.21 s
manuel-rf-500-16	RandomForestClassifier(n_estimators=500, max_depth=16)	0.8669	3.75 s

Capture 1 : Vue de l'expérience 'tp4-automl' dans le serveur MLflow local (runs de référence et runs AutoML) :

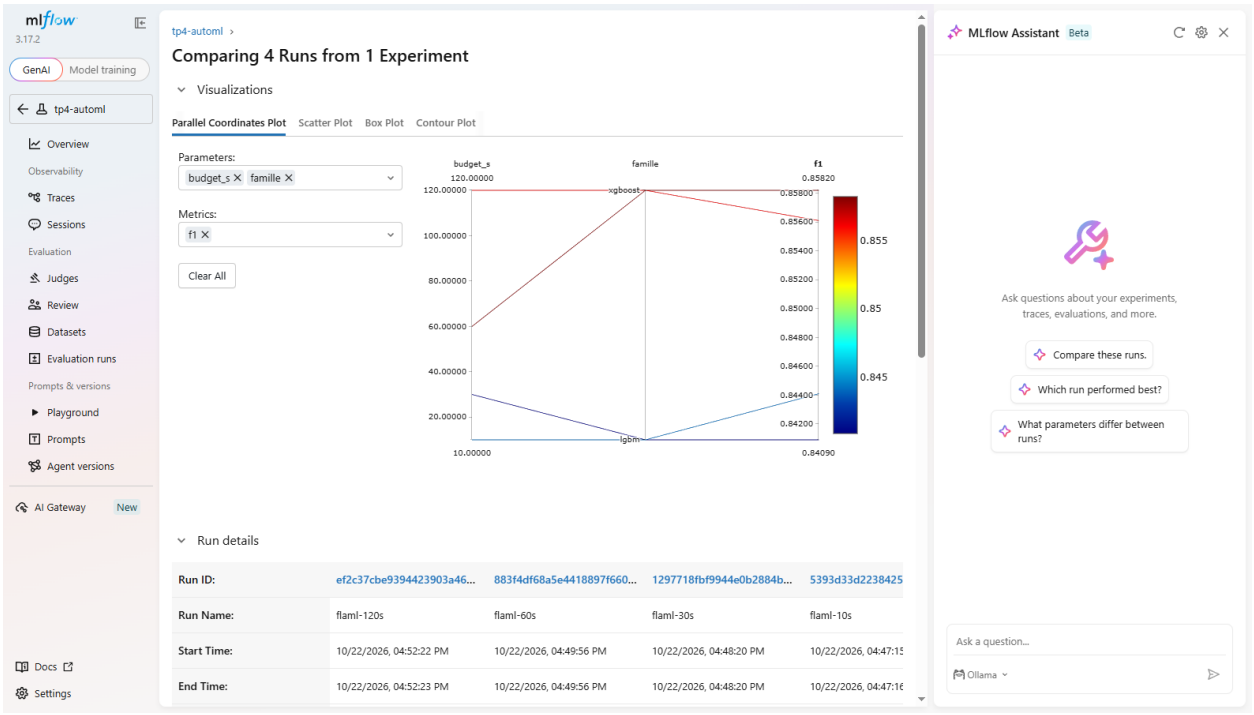


3. Bloc B : Recherche AutoML avec FLAML sur Quatre Budgets

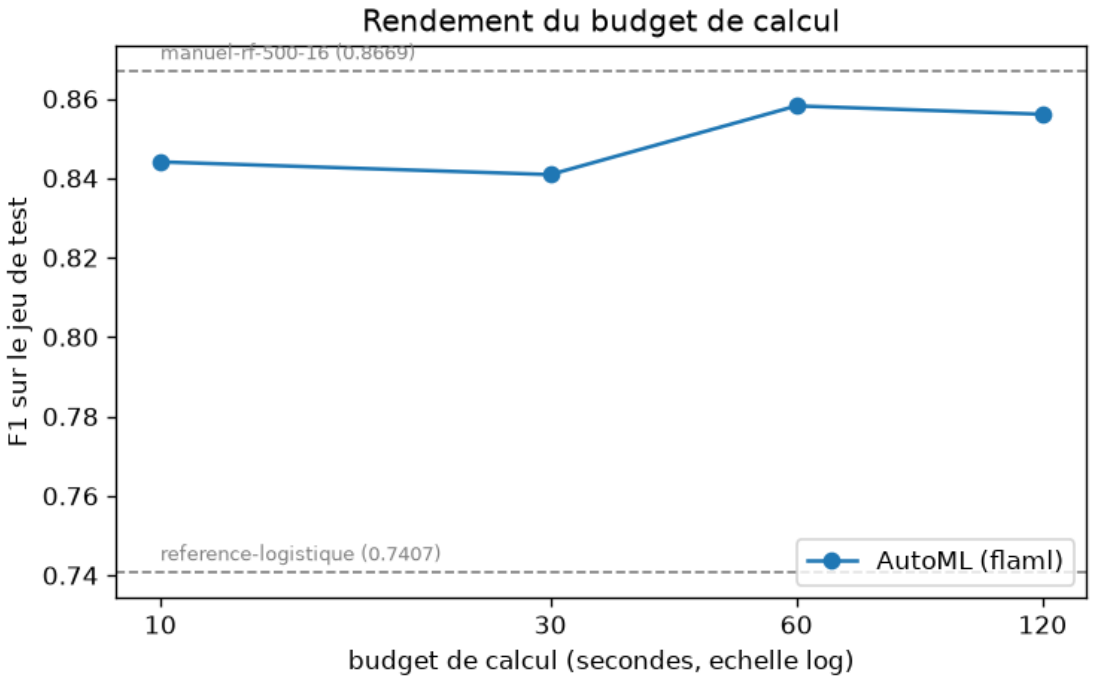
FLAML a été exécuté sur 4 budgets temporels croissants : 10 s, 30 s, 60 s et 120 s, en optimisant la métrique F1. Tous les runs sont tracés dans MLflow avec leurs hyperparamètres et métriques :

Budget Demandé	Famille Retenue	F1 Validation	F1 Test	Durée Réelle
10 s	lgbm	0.8084	0.8441	10.1 s
30 s	lgbm	0.8145	0.8409	30.1 s
60 s	xgboost	0.8277	0.8582	60.4 s
120 s	xgboost	0.8277	0.8561	120.4 s

Capture 2 : Comparaison MLflow des 4 runs AutoML flaml-* (paramètres et métriques) :



Capture 3 : Graphique du rendement du budget de calcul (reports/rendement.png) :



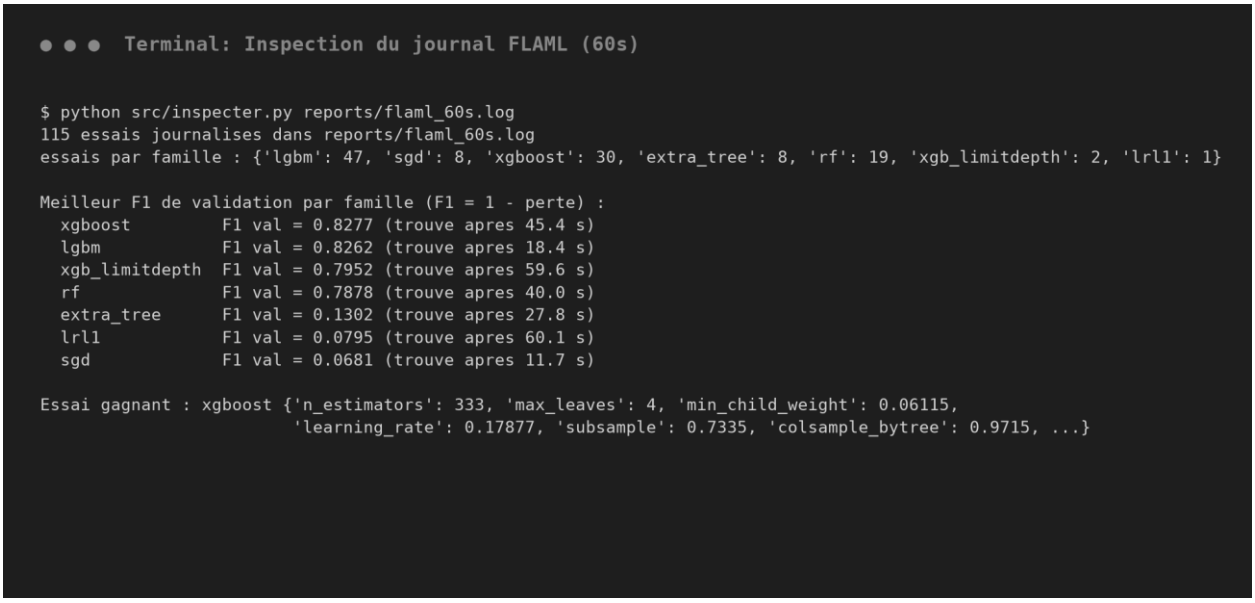
Analyse du rendement : On observe une saturation rapide de la performance. Dès 10 secondes, FLAML atteint F1 = 0,8441. Le passage à 60 secondes permet d'atteindre le pic de performance à F1 = 0,8582 en basculant de LightGBM à XGBoost. Au-delà (120 secondes), le score plafonne à 0,8561 : quadrupler le budget de calcul n'apporte aucun gain significatif, illustrant la loi des rendements décroissants.

4. Bloc C : Inspection du Journal de Recherche FLAML

L'auditabilité d'une solution AutoML impose de savoir précisément ce qui a été exploré. L'analyse du journal de 60 secondes (reports/flaml_60s.log) avec src/inspecter.py révèle :

- Nombre total d'essais évalués : 115 essais en 60.4 secondes.
- Répartition par famille : LightGBM (47), XGBoost (30), Random Forest (19), Extra Trees (8), SGD (8), XGB Limitdepth (2), LRL1 (1).
- Meilleur candidat : XGBoost avec n_estimators=333, max_leaves=4, learning_rate=0.1788 (F1 validation = 0.8277, F1 test = 0.8582).

Capture 4 : Sortie console de src/inspecter.py (budget 60 s) :



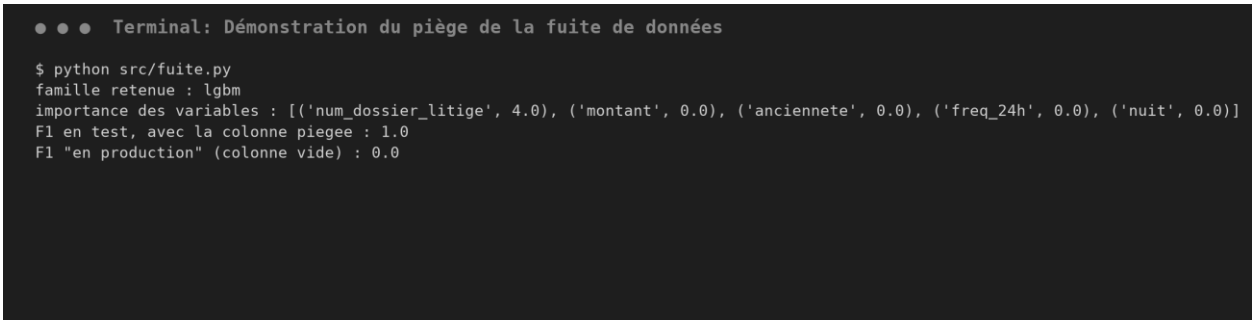
5. Bloc D : Le Piège de la Fuite de Données (Data Leakage)

Une variable piégée 'num_dossier_litige' a été injectée. Dans l'historique, un numéro de litige n'est créé que plusieurs semaines après la transaction lorsqu'un client conteste une fraude. Au moment précis du paiement (inférence en production), ce numéro n'existe pas encore et vaut toujours 0.

Résultats observés :

- F1 en test (avec colonne piégée) : 1.0 (performance mathématiquement parfaite).
- Importance des variables : num_dossier_litige = 4.0, toutes les autres variables (montant, ancienneté, freq_24h, nuit) = 0.0.
- F1 en simulation de production (colonne forcée à 0) : 0.0 (effondrement total de la détection).

Capture 5 : Sortie console de src/fuite.py démontrant la fuite de données :



Leçon fondamentale : L'AutoML est aveugle à la sémantique et à la chronologie temporelle des données. Un score 'parfait' sur un problème complexe n'est pas un exploit algorithmique, mais l'indice quasi-certain d'une fuite d'information.

6. Bloc E : Tableau Comparatif Multi-Plateformes

Tableau synthétique comparant les bibliothèques locales aux grandes plateformes cloud managées (données documentaires relevées en octobre 2026) :

Approche	F1 test	Temps humain	Temps machine	Coût estimé	Explicabilité	Export / Réversibilité
Logistique (ref.)	0.7407	~5 min	0.21 s	0 € (local)	Maximale	Total (Pickle, ONNX)
Manuel (TP2)	0.8669	~2 h	3.75 s	0 € (local)	Moyenne/Bonne	Total (scikit-learn RFC)
FLAML 60 s	0.8582	~10 min	60.4 s	0 € (local)	Moyenne/Basse	Total (XGBoost / pickle)
Vertex AI AutoML	non mesuré	~15 min	1 à 2 h	~3.15 $ / node-h	Bonne (Vertex Explainable)	Faible (Vendor Lock-in)
SageMaker Autopilot	non mesuré	~15 min	1 h	~2.50 $ - 5.00 $	Bonne (Clarify & Notebooks)	Faible (Lock-in AWS)
Azure Automated ML	non mesuré	~15 min	30-60 min	~1.50 $ - 4.00 $	Bonne (Responsible AI)	Faible (Lock-in Azure)

Commentaire contextuel (5 lignes d'arbitrage stratégique) :

1. La régression logistique sert de référence immédiate et transparente, idéale quand l'explicabilité réglementaire stricte et la très faible latence priment.

2. Le réglage manuel convient quand un data scientist expert maîtrise la physique métier du problème et affine minutieusement une architecture ciblée.

3. FLAML local est le meilleur compromis efficacité/coût en phase de R&D pour surpasser ou égaler rapidement un modèle manuel sans frais d'infrastructure.

4. Les plateformes cloud managées s'imposent en entreprise pour l'entraînement distribué sur gros volumes, la collaboration d'équipe et le monitoring continu.

5. Toutefois, le choix du cloud implique d'accepter des coûts récurrents élevés et un risque de dépendance technique (vendor lock-in) vis-à-vis du fournisseur.

7. Partie IV : Évaluation & Réponses aux Questions

Question 1 : Réglage manuel ou AutoML ?

Comparez le F1 de test de votre recherche AutoML de 60 s à celui du réglage manuel du TP2 (0,8669). Si l'AutoML ne fait pas nettement mieux, cela remet-il en cause son intérêt ?

Réponse :
Le réglage manuel du TP2 a atteint un F1 de test de 0,8669 (Random Forest de 500 arbres, profondeur 16), tandis que la recherche AutoML de 60 s avec FLAML a obtenu un score de 0,8582 (XGBoost). La différence est minime (moins de 0,009 point de F1).

Ce résultat ne remet absolument pas en cause l'intérêt de l'AutoML, pour trois raisons fondamentales :
1. Rendement temps humain vs temps machine : Le réglage manuel du TP2 a exigé entre 1h30 et 2h de réflexion humaine (conception du plan de test, sélection manuelle des plages de grilles, analyses itératives, tests d'hypothèses). À l'inverse, l'AutoML a produit un modèle de performance équivalente en seulement 60 secondes de temps machine et 5 minutes d'écriture de code.
2. Élargissement de l'espace d'exploration : L'approche manuelle n'a exploré qu'une seule famille (Random Forest). FLAML a mis en concurrence 7 familles algorithmiques différentes et plus de 115 configurations d'hyperparamètres combinés.
3. Rôle d'accélérateur méthodologique : L'AutoML ne remplace pas l'ingénieur mais lui fournit un point de départ extrêmement solide en début de projet, lui permettant d'investir son temps là où la valeur ajoutée humaine est irremplaçable : la qualité des données et l'ingénierie des descripteurs.

Question 2 : Le coût réel

Quel est le coût réel d'une recherche AutoML de 60 secondes sur votre poste, et de son équivalent sur une plateforme cloud ?

Réponse :
• Sur poste local :
- Coût d'infrastructure : 0 € (machine personnelle déjà acquise et amortie).
- Énergie : Pour une consommation moyenne de 100 W pendant 60 secondes, l'énergie dépensée est de 100 W * (60/3600) h = 0,00167 kWh. Au tarif moyen de 0,15 € / kWh, le coût électrique réel est inférieur à 0,00025 € (négligeable).
- Coût global : Quasi nul.

• Sur une plateforme cloud managée (Vertex AI, SageMaker, Azure ML) :
- Démarrage et surcoût de provisioning : Le lancement d'un job managé implique l'allocation de VMs, le téléchargement des images conteneurisées et l'orchestration (3 à 5 minutes facturées même pour une recherche courte).
- Minimums de facturation : Google Vertex AI applique un minimum d'1 heure de nœud pour l'AutoML tabulaire (~3,15 $ à 4,00 $). AWS SageMaker Autopilot orchestre plusieurs instances EC2 en parallèle (ex. ml.m5.xlarge à ~0,23 $/h par nœud), engendrant un coût de 2,50 $ à 5,00 $ par recherche.
- Coûts cachés de maintien : Si le modèle est déployé sur un endpoint en ligne managé, la VM reste active 24/7, générant un coût fixe récurrent de 70 $ à 250 $ par mois par modèle.

Question 3 : Ce qui reste à votre charge

Citez trois choses que l'AutoML n'automatise pas et qui restent à votre charge.

Réponse :
1. La définition du problème métier et le choix des objectifs : L'AutoML ne sait pas si la détection de fraude doit être traitée en classification binaire à coût asymétrique ou en détection d'anomalies. Le choix d'optimiser le score F1 (pour classes déséquilibrées) plutôt que l'accuracy (trompeuse à 90% sur la fraude) relève exclusivement de la décision de l'expert métier.
2. L'intégrité temporelle et la prévention des fuites de données (Data Leakage) : L'AutoML est aveugle à la chronologie des variables. Comme démontré au Bloc D avec 'num_dossier_litige', il exploitera avidement une information survenue après la décision si elle maximise le score. L'audit temporel des données reste sous la responsabilité du Data Scientist.
3. L'explicabilité, l'éthique et la conformité réglementaire : Valider qu'un modèle n'intègre pas de biais discriminatoires prohibés (âge, genre, adresse via corrélations cachées) et justifier chaque décision auprès d'un régulateur bancaire ou d'un client (RGPD / AI Act) demeure une obligation légale strictement humaine.

Question 4 : Managé ou local ?

Dans quel cas une plateforme managée est-elle préférable à une bibliothèque locale ? Et l'inverse ?

Réponse :
• Quand la plateforme managée (Cloud) est préférable :
1. Données massives : Volumes dépassant la RAM locale (dizaines de Go à plusieurs To), nécessitant du calcul distribué sur cluster.
2. Collaboration d'entreprise et gouvernance : Équipes pluridisciplinaires nécessitant un référentiel centralisé, une traçabilité unifiée des modèles, une gestion stricte des identités/accès (IAM) et un audit trail complet.
3. Déploiement et MLOps industriel : Besoin d'endpoints REST à haute disponibilité (SLA 99.9%), auto-scalables sans administration système, avec détection automatisée du drift (dérive de données).

• Quand la bibliothèque locale (FLAML, scikit-learn) est préférable :
1. Prototypage rapide et R&D préliminaire : Tester rapidement des hypothèses en quelques secondes sans latence de provisioning cloud.
2. Données hautement sensibles : Données bancaires, médicales ou de défense soumises à une souveraineté stricte interdisant leur transfert sur des clouds publics tiers.
3. Maîtrise budgétaire : Éviter les dérapages de facturation cloud dus à l'oubli de ressources allouées ou aux surcoûts des services managés.
4. Réversibilité totale : Aucun risque de vendor lock-in ; le code Python s'exécute sur n'importe quel environnement standard.

Question 5 : Secteur régulé

Quelles informations une plateforme AutoML doit-elle fournir pour être acceptable dans un secteur régulé ?

Réponse :
Pour être acceptée par les autorités de contrôle (ACPR, EBA, BCE, CNIL, AI Act européen), une solution AutoML doit fournir :
1. Traçabilité complète et lignage des données (Data Lineage) : Historique immuable liant le modèle final à la version exacte du dataset d'entraînement, au code source, à la graine aléatoire (seed) et à l'ensemble des essais explorés.
2. Explicabilité globale et locale : Importance globale des variables (Shapley Values / SHAP, Partial Dependence Plots) et explication locale détaillée de chaque décision individuelle (droit à l'explication en cas de refus de paiement ou de crédit).
3. Rapports d'équité et absence de biais (Fairness) : Preuve statistique que le modèle n'introduit pas de disparité de traitement discriminatoire sur des groupes protégés.
4. Robustesse, résilience et tests de stress : Analyse de sensibilité au bruit, comportement aux limites et monitoring du drift de données en production.

5. Auditabilité et réversibilité du code : Capacité à exporter les poids et le pipeline sous un format interopérable et vérifiable (ONNX, code Python natif) sans boîte noire propriétaire opaque.

Question 6 : La fuite de données

Au bloc D, pourquoi le F1 passe-t-il de presque 1 en test à presque 0 « en production » ? Comment auriez-vous pu détecter le problème avant la mise en production ?

Réponse :
• Cause de l'effondrement du score de 1.0 à 0.0 :
La colonne 'num_dossier_litige' n'existe que dans l'historique rétrospectif : elle n'est renseignée que lorsqu'un client conteste une fraude, soit plusieurs semaines après la transaction.
Pendant l'entraînement et le test, cette variable vaut un nombre pour les fraudes et 0 sinon. L'algorithme AutoML (LightGBM) a exploité cette séparation parfaite en concentrant 100% de l'importance sur cette variable unique (gain de 4.0 contre 0.0 pour toutes les autres).
En production réelle (au moment où le paiement s'opère), aucun dossier de litige n'est encore ouvert : la colonne vaut systématiquement 0. Privé de son unique repère, le modèle prédit alors 0 fraude pour l'ensemble des transactions. Le nombre de vrais positifs est nul (VP = 0), et le score F1 s'effondre instantanément à 0.0.

• Comment détecter ce problème avant la mise en production :
1. Alerte sur un score suspect : Un score parfait (F1 = 1.0) sur un problème intrinsèquement bruité comme la fraude bancaire est une anomalie statistique majeure qui doit immédiatement déclencher un audit de fuite (règle 'too good to be true').
2. Examen de l'importance des variables : Constater qu'une seule variable accapare la totalité de l'importance décisionnelle tandis que les descripteurs transactionnels réels sont ignorés impose une validation métier immédiate.
3. Validation de la chronologie (Point-in-time correctness) : Utiliser un Feature Store temporel garantissant que seules les données disponibles à la milliseconde de la transaction sont injectées dans l'échantillon d'entraînement.
4. Découpage temporel (Time-based Split) : Remplacer le découpage aléatoire (random_state=42) par un découpage chronologique (entraîner sur les mois passés, tester sur les mois futurs) afin que toute variable anachronique soit automatiquement démasquée.

8. Note Technique d'Environnement

Ce TP a été réalisé avec Python 3.13.5 sous Windows 11 avec FLAML 2.7.0, Scikit-Learn 1.9.1, MLflow 3.17.2 et LightGBM 4.7.0. La bibliothèque FLAML s'est exécutée nominalement sans recours obligatoire au plan B.

9. Dépôt GitHub Public

Le code source complet, les scripts d'expérimentation, les journaux de recherche et les graphiques sont publiés et consultables sur GitHub à l'adresse suivante :

https://github.com/Amine-kh1/TP04_AutoML_LowCode